

# Chapitre 03\_Partie2\_Architecture Oracle Database

# Rappel\_Composants de l'architecture Oracle

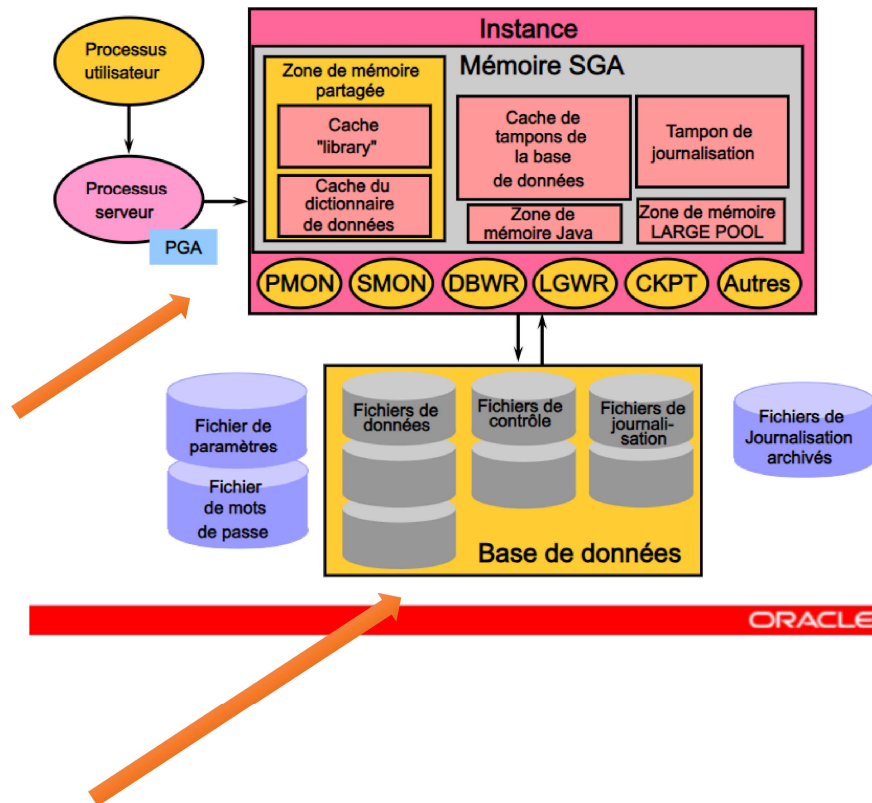
L'architecture Oracle comporte plusieurs composants principaux qui seront présentés plus loin dans ce chapitre.

- **Serveur Oracle** : Un serveur Oracle comporte plusieurs **fichiers**, **processus** et **structures mémoire**, mais ces éléments ne sont pas tous utilisés dans le traitement des **instructions SQL**. Certains de ces éléments améliorent les **performances** de la base de données, permettent de **recupérer** la base en cas d'incident logiciel ou matériel ou exécutent d'autres tâches nécessaires à la **gestion de la base**. Le serveur Oracle est constitué d'une instance Oracle et d'une base Oracle. **Serveur Oracle = Instance Oracle + Base de données Oracle**

1. **Instance Oracle** : Une instance Oracle est une combinaison des **processus d'arrière-plan** et des **structures mémoire**. Pour accéder aux données de la base, il est nécessaire de **démarrer l'instance**. A chaque démarrage d'instance, une mémoire **SGA (System Global Area)** est allouée et des **processus d'arrière-plan Oracle** sont lancés. Ces processus exécutent des fonctions pour le compte du **processus appelant (processus utilisateur)**. Ils regroupent des fonctions qui, sinon, seraient gérées par plusieurs programmes Oracle exécutés par chaque utilisateur. Les **processus d'arrière-plan** effectuent des opérations d'entrée/sortie et surveillent d'autres processus Oracle afin de permettre un plus grand **parallélisme** et d'améliorer les **performances** et la **fiabilité**.

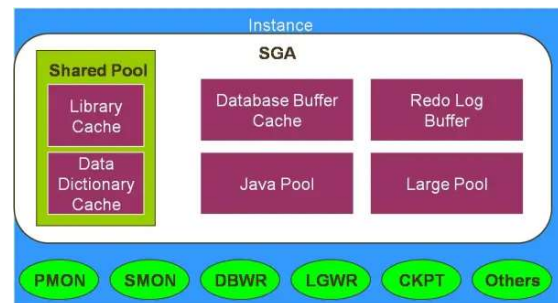
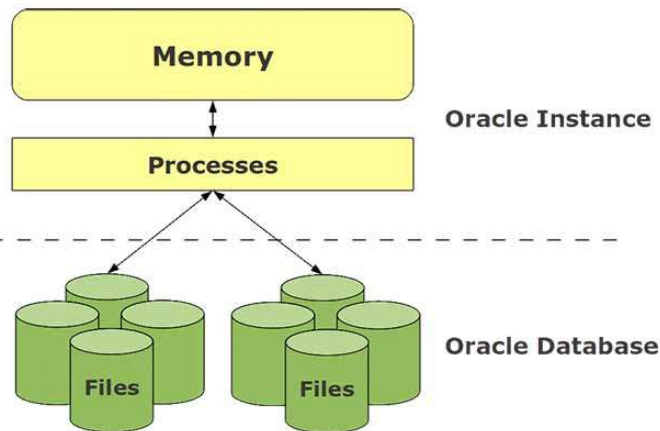
2. **Base de données Oracle** : La base Oracle est constituée de fichiers du système d'exploitation, appelés **fichiers de base de données**. Ces fichiers constituent l'**espace de stockage physique** des données de la base. Ils permettent de maintenir la **cohérence** des données et peuvent être **recupérés** en cas d'**échec de l'instance**.

## Présentation des principaux composants



# Rappel\_Composants de l'architecture Oracle – Instance Oracle

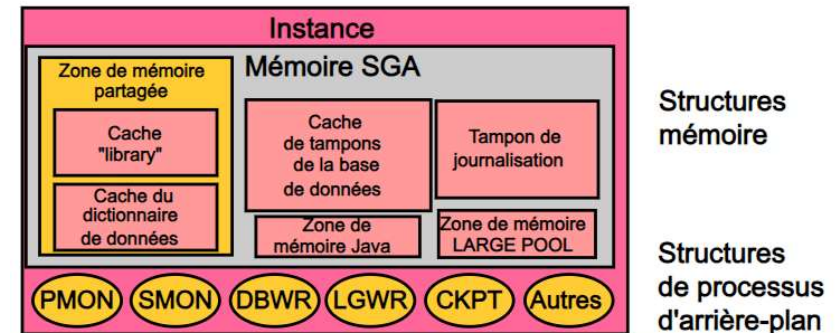
- **L'instance** est l'ensemble de **structures mémoires** et de **processus** qui assurent l'**accès** et la **gestion** d'une **base de données**. Le **fichier de paramètres** est utilisé pour configurer l'instance lors de son démarrage.
- **Une instance** est **identifiée** à l'aide de méthodes propres à chaque **système d'exploitation**. Elle ne peut **ouvrir** et **utiliser** qu'une seule base à la fois.



## Instance Oracle

Une instance Oracle :

- permet d'accéder à une base de données Oracle,
- n'ouvre qu'une seule base de données,
- est constituée de structures de processus d'arrière-plan et de structures mémoire.





# Instance Oracle : Processus

Les **processus Oracle** permettent aux différentes composantes du serveur d'**interagir** et d'**échanger** entre elles ainsi qu'avec l'utilisateur :

- Il existe des **processus utilisateurs** du côté des clients, un ou plusieurs **processus serveurs** du côté du serveur assurant la communication avec les clients;
- Plusieurs **processus d'arrière plan** qui assurent le bon fonctionnement de l'instance.

## Structure de processus

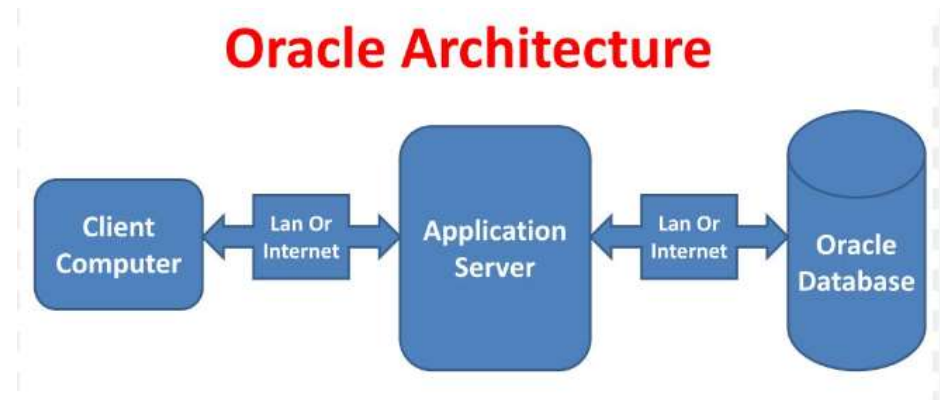
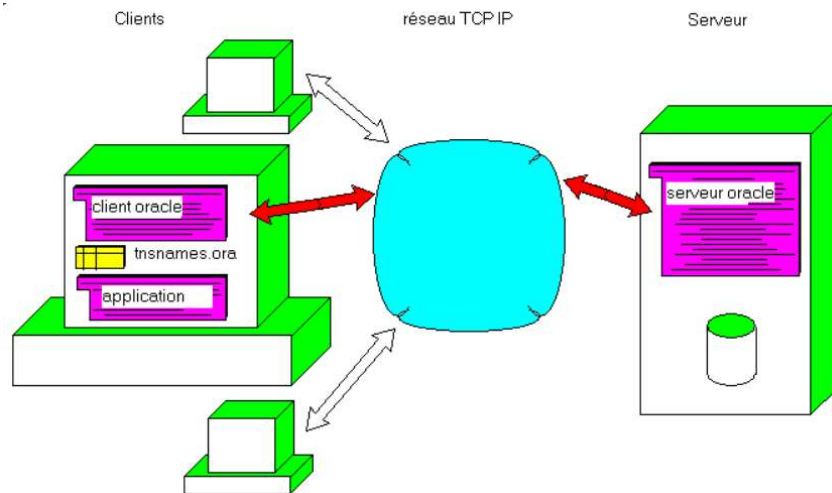
Oracle utilise différents types de processus :

- le processus utilisateur, qui est démarré au moment où un utilisateur de la base de données tente de se connecter au serveur Oracle,
- le processus serveur, qui établit la connexion à l'instance Oracle et démarre lorsqu'un utilisateur ouvre une session,
- les processus d'arrière-plan, lancés au démarrage d'une instance Oracle.

# Instance Oracle : Processus

## Les processus-utilisateur et le processus-serveur :

- L'interaction entre les **clients** et le **serveur de base de données** est assurée par le **processus utilisateur** du **côté client** et par le **processus serveur** du côté du **serveur**. En effet, lorsqu'un utilisateur démarre une **application** (**SQL\*Plus**, **Oracle Enterprise Manager**, une application **Developer Forms** etc.), Oracle démarre un **processus utilisateur** que ce soit sur sa machine distante s'il s'agit d'une **architecture client-serveur** ou sur un **serveur middle-tier** sur une **architecture multi-tier**.
- Par la suite, une **connexion** entre le **processus utilisateur** et l'**instance** (**processus serveur**) est initiée et maintenue. Une fois la connexion établie, l'utilisateur ouvre une **session** en s'identifiant (saisie de son nom d'utilisateur et de son mot de passe). Il est à noter que plusieurs sessions peuvent être ouvertes en même temps, ces sessions peuvent être ouvertes par le même utilisateur. Le paramètre qui détermine le **nombre maximum de sessions ouvertes en même temps** est le paramètre **SESSIONS**.

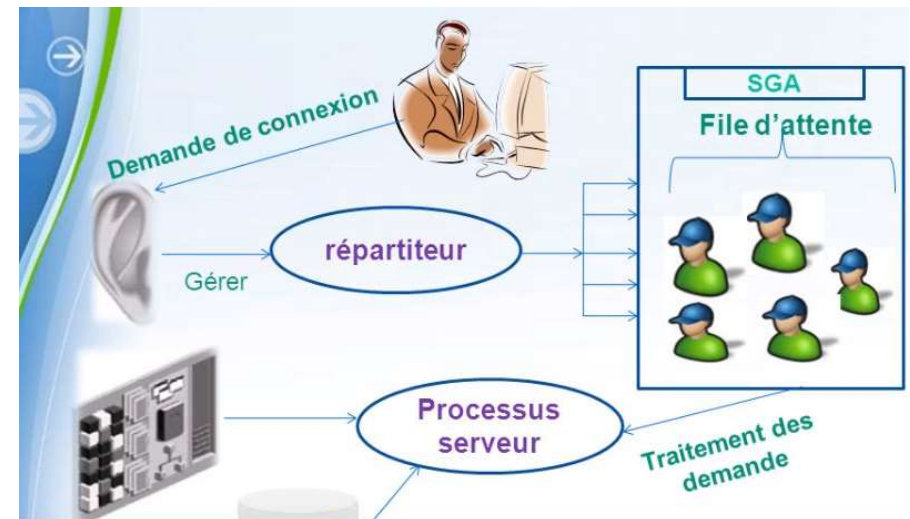
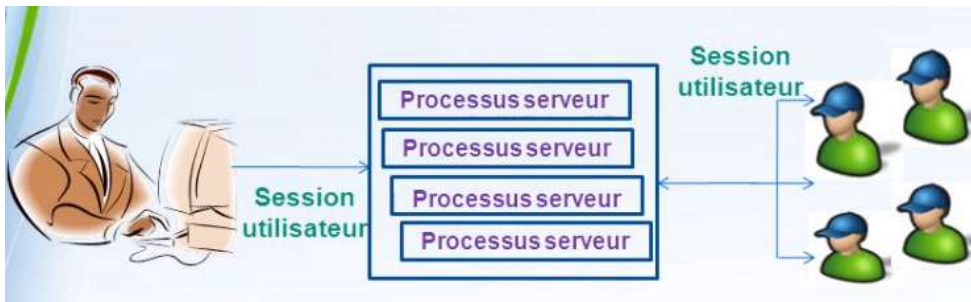




# Instance Oracle : Processus

## Les processus-utilisateur et le processus-serveur (suite) :

- Un serveur base de données est soit configuré en mode **dédié**, soit en mode **partagé**. Lorsque le serveur est configuré en **mode dédié**, Oracle crée sur le serveur **pour chaque utilisateur** (et donc pour chaque processus utilisateur) **un processus serveur**. **En mode partagé**, **plusieurs processus utilisateurs** peuvent partager un **seul et unique processus serveur**. Le **mode par défaut** est le **mode dédié**, pour activer le mode partagé, il suffit de modifier le paramètre **SHARED\_SERVERS** dont la **valeur par défaut** est **0**. La nouvelle valeur indiquerait le **nombre de processus serveurs partagés** démarrés lorsque l'instance est lancée.
- Rappelons qu'une **zone de mémoire PGA** est créée pour chaque **processus serveur**, elle inclut entre autres les **variables hôtes** et les **variables de session** utilisées dans les requêtes/programmes envoyés par l'utilisateur.



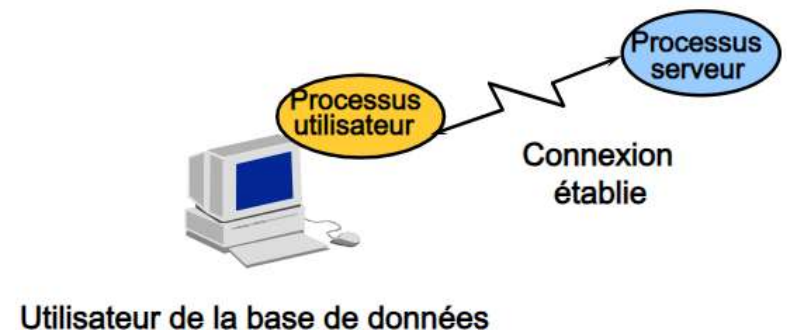
# Instance Oracle : Processus → Processus utilisateur

## Processus-utilisateur :

- Un utilisateur de base de données qui doit demander des informations contenues dans la base doit d'abord établir une **connexion** avec le serveur Oracle.
- La demande de connexion s'effectue via un outil d'interface de base de données (**SQL\*Plus**, par exemple) et le lancement du **processus utilisateur**.
- Ce dernier n'entre pas directement en interaction avec le serveur Oracle.
- Il génère des appels via l'interface **UPI (User Program Interface)** qui crée une **session** et démarre un **processus serveur**.

## Processus utilisateur

- Programme qui demande une interaction avec le serveur Oracle.
- Ce processus doit d'abord établir une connexion.
- Il n'entre pas directement en interaction avec le serveur Oracle.



# Instance Oracle : Processus → Processus serveur

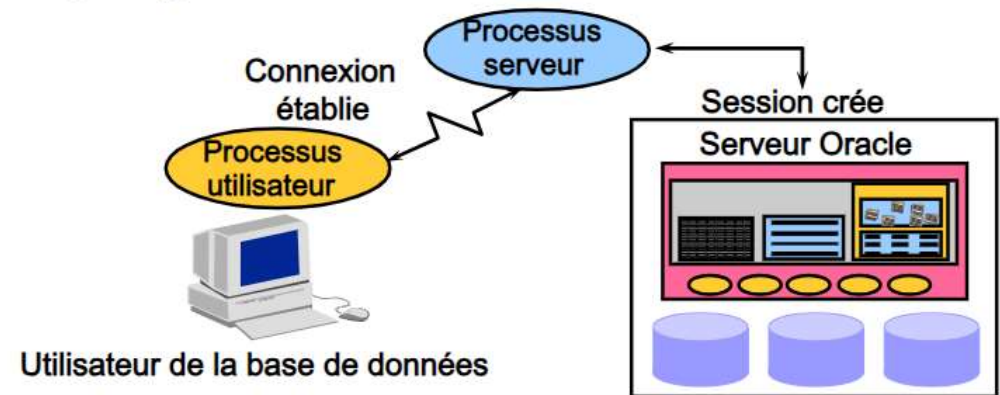
## Processus-serveur :

Lorsque l'utilisateur a établi une **connexion**, un **processus serveur** démarre pour traiter les demandes du **processus utilisateur**. Un **processus serveur** peut être **dédié** ou **partagé**.

- Dans un **environnement serveur dédié**, le processus serveur traite la demande d'un **seul processus utilisateur**. Le processus serveur prend fin lorsque le processus utilisateur se déconnecte.
- Dans un **environnement serveur partagé**, le processus serveur traite les demandes de **plusieurs processus utilisateur**. Le processus serveur communique avec le serveur Oracle à l'aide de l'interface **OPI** (Oracle Program Interface).

## Processus serveur

- Programme qui entre directement en interaction avec le serveur Oracle.
- Il répond aux appels générés et renvoie les résultats.
- Il peut s'agir d'un serveur dédié ou d'un serveur partagé.





# Instance Oracle : Processus → Processus arrière plan

## Les processus d'arrière plan :

- L'architecture Oracle possède **cinq processus d'arrière-plan obligatoires** qui seront présentés plus loin dans ce chapitre.
- Oracle possède, en outre, un certain nombre de processus d'arrière-plan **facultatifs** qui sont démarrés via l'exécution de l'option correspondante.
- Ces processus d'arrière plan sont utilisés par Oracle afin de **maximiser la performance de l'instance**.
- La majorité des **processus d'arrière plan** sont **lancés** lorsque **l'instance démarre**, certains d'entre eux pourraient être lancés après, lorsque **l'instance en aura besoin**.
- Il est à noter que certains processus fonctionnent en **plusieurs exemplaires**, c'est pour cela que **leur nom** finit par **n** qui signifie le numéro de l'exemplaire du processus.

## Processus d'arrière-plan

Gèrent et appliquent les relations entre les structures physiques et les structures mémoire.

- **Processus d'arrière-plan obligatoires**

|        |      |      |
|--------|------|------|
| – DBWn | PMON | CKPT |
| – LGWR | SMON |      |

- **Processus d'arrière-plan facultatifs**

|        |      |      |
|--------|------|------|
| – ARCn | LMDn | RECO |
| – CJQ0 | LMON | Snnn |
| – Dnnn | Pnnn |      |
| – LCKn | QMn  |      |



# Instance Oracle : Processus → Processus arrière plan

## Les processus d'arrière plan :

La liste suivante contient une partie des **processus d'arrière-plan facultatifs** :

- ❖ **RECO** : Processus de récupération
- ❖ **QMNn** : Advanced Queuing
- ❖ **ARCn** : Processus d'archivage
- ❖ **LCKn** : RAC Lock Manager–Verrous d'instance
- ❖ **LMON** : RAC DLM Monitor–Verrous globaux
- ❖ **LMDn** : RAC DLM Monitor–Verrous à distance
- ❖ **CJQ0** : Processus d'arrière-plan Coordinator Job Queue
- ❖ **Dnnn** : Répartiteur (Dispatcher)
- ❖ **Snnn** : Serveur partagé
- ❖ **Pnnn** : Processus esclave "Parallel Query"

## Processus d'arrière-plan

Gèrent et appliquent les relations entre les structures physiques et les structures mémoire.

- **Processus d'arrière-plan obligatoires**

|        |      |      |
|--------|------|------|
| – DBWn | PMON | CKPT |
| – LGWR | SMON |      |

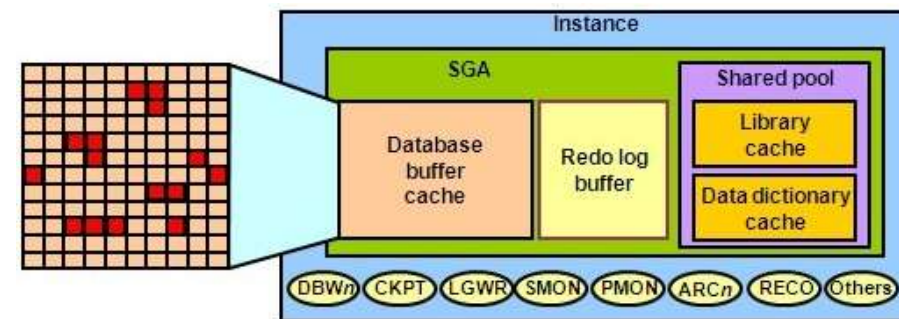
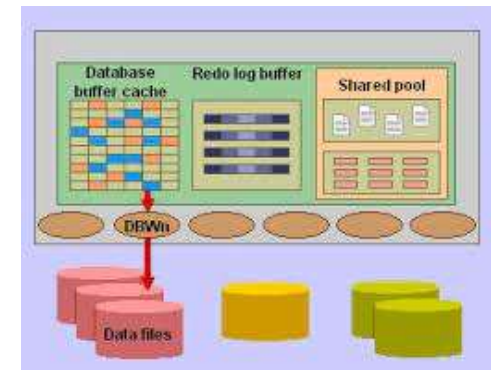
- **Processus d'arrière-plan facultatifs**

|        |      |      |
|--------|------|------|
| – ARCn | LMDn | RECO |
| – CJQ0 | LMON | Snnn |
| – Dnnn | Pnnn |      |
| – LCKn | QMNn |      |

# Instance Oracle : Processus → Processus database writer (DBWn)

## Processus database writer (DBWn) :

- **DBWn** est le processus qui écrit le contenu du **cache des données** dans les **fichiers de données** (Database Writer). Le **DBW** écrit les tampons (buffers) modifiés, dits *dirty*, du cache de données dans les fichiers de données sur le disque.
- Il garantit qu'un nombre suffisant de mémoires **tampon libres** (tampons qui peuvent être écrasés lorsque les **processus serveur** doivent lire **des blocs dans les fichiers de données**) est disponible dans le **cache de tampons** de la base de données.
- Les **performances** de la base sont améliorées puisque les **processus serveur** n'effectuent les modifications que dans le **cache de tampons de la base de données**.
- Un administrateur Oracle peut spécifier jusqu'à **20 exemplaires** de **DBW**, de DBW0 jusqu'à DBW9 et de DBWa jusqu'à DBWj si la base de données connaît une forte **charge transactionnelle**. Il est à noter que les exemplaires des **DBW** sont **inutiles** dans le cadre de **serveurs monoprocesseurs**.
- Le paramètre d'initialisation **DB\_WRITER\_PROCESSES** spécifie le **nombre d'exemplaires** du **DBW**. Le **maximum** est de **20**, si non spécifié, Oracle se charge d'affecter le **nombre d'exemplaires** qui convient selon le **nombre de processeurs** du serveur hébergeant la base.

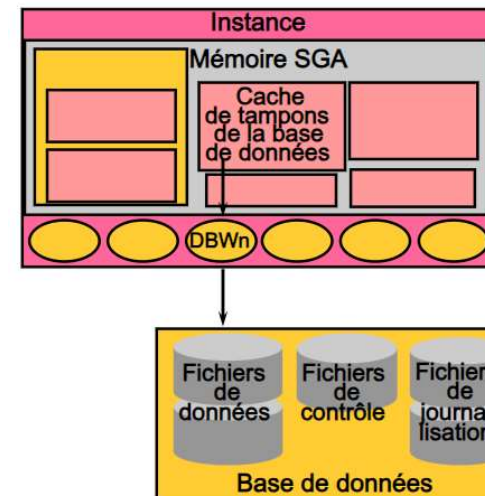


# Instance Oracle : Processus → Processus database writer (DBWn)

## Processus database writer (DBWn) (suite) :

- La tâche du **DBW** est de « nettoyer » le **cache des données** en écrivant les blocs **dirty** les **moins récemment utilisés** dans le **fichier des données** de manière à ce que le(s) **processus serveur** trouve(nt) aisément des tampons **clean** pour y charger les blocs de données qu'il faut (à la suite d'une interrogation utilisateur) à partir des fichiers de données.
- Il faut savoir que l'écriture des blocs **dirty** par le **DBW** est **différée**, c-à-d, qu'Oracle n'écrit pas les blocs modifiés automatiquement après l'exécution de la requête de mise à jour, et pas forcément après la confirmation (COMMIT) de la transaction en question.
- En d'autres termes, ce n'est pas l'exécution de la transaction ni sa confirmation qui déclenchent le fonctionnement du **DBW**.

## Processus database writer (DBWn)



DBWn écrit dans les cas suivants :

- point de reprise
- seuil des tampons "dirty" atteint
- aucune mémoire tampon disponible
- temps imparti dépassé
- demande de ping RAC
- tablespace hors ligne
- tablespace en lecture seule
- DROP ou TRUNCATE sur une table
- BEGIN BACKUP sur un tablespace

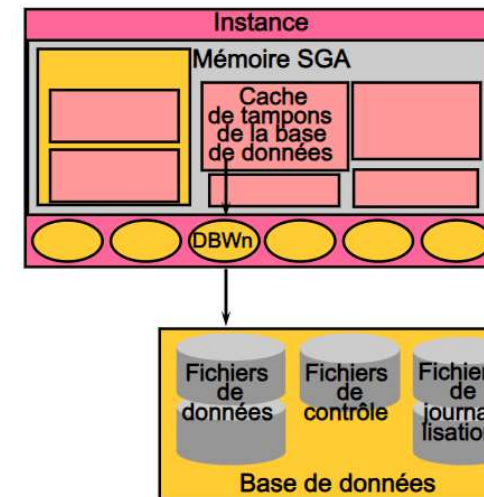
# Instance Oracle : Processus → Processus database writer (DBWn)

## Processus database writer (DBWn) (suite) :

- Ce processus est plutôt déclenché par les événements suivants (suite) :

- ❖ Après une certaine période pour faire avancer le **point de reprise**. Le point de reprise est la position dans les **fichiers journaux** à partir de laquelle l'instance est récupérable. Cette position est déterminée par le plus ancien tampon **dirty** dans le cache de données.
- ❖ Le nombre de tampons "**dirty**" a atteint une **valeur seuil**.
- ❖ Un processus balaie un certain nombre de blocs à la recherche de mémoires **tampon libres et n'en trouve pas**.
- ❖ Le **temps imparti** est dépassé.
- ❖ Une demande de ping est émise dans l'environnement Real Application Clusters (RAC).
- ❖ Un **tablespace** normal ou temporaire est mis **hors ligne**.
- ❖ Un **tablespace** est mis en **lecture seule**.
- ❖ Une **table** est **supprimée** ou **vidée**.
- ❖ La commande suivante est exécutée : **ALTER TABLESPACE** nom du tablespace **BEGIN BACKUP**.

## Processus database writer (DBWn)



DBWn écrit dans les cas suivants :

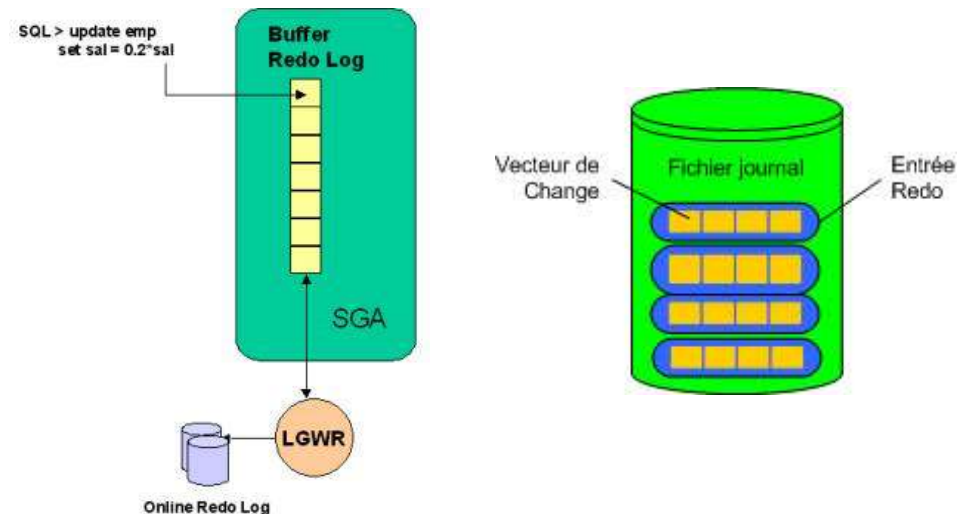
- point de reprise
- seuil des tampons "dirty" atteint
- aucune mémoire tampon disponible
- temps imparti dépassé
- demande de ping RAC
- tablespace hors ligne
- tablespace en lecture seule
- DROP ou TRUNCATE sur une table
- BEGIN BACKUP sur un tablespace



# Instance Oracle : Processus → Processus LGWR (Log Writer)

## Processus LGWR (Log Writer) :

- **LGWR** est le processus qui écrit le contenu (les **entrées Redo**) du **redo log buffer** sur les **fichiers de journalisation** de manière **séquentielle**.
- Une fois les **entrées redo** sont écrites sur le disque, le **processus serveur** peut utiliser cet espace (en écrasant les entrées déjà écrites) pour y stocker les informations des **nouvelles mises à jour** toujours sous la forme d'**entrées Redo**.
- Le **LGWR** est déclenché par les événements suivants :
  - 1) Lorsqu'un **tiers** (1/ 3) du **tampon de journalisation** est occupé.
  - 2) Lorsque le tampon de journalisation contient **plus d'un mégaoctet** de modifications enregistrées.
  - 3) Toutes les **trois secondes**
  - 4) L'utilisateur confirme une transaction par un **COMMIT**. Remarquez qu'un **COMMIT** déclenche **uniquement** le **LGWR**. Après un **COMMIT**, les entrées Redo (du log buffer) en relation avec la transaction confirmée sont toutes écrites sur les fichiers journaux et un message de confirmation de la mise à jour est renvoyé à l'utilisateur. Cette notion d'écriture rapide du contenu du log buffer est dite ***fast commit***.

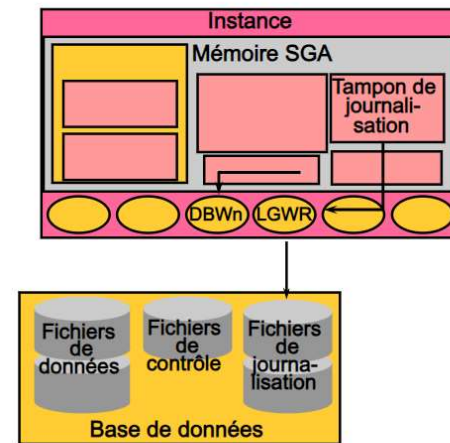


# Instance Oracle : Processus → Processus LGWR (Log Writer)

## Processus LGWR (Log Writer) (suite) :

- 5) Avant que le **DBWn** ne se mette à écrire les **blocs modifiés non validés** (non « committés ») sur disque. En effet, lorsque le **DBWn** s'apprête à écrire des blocs **dirty** non validés, il faut que toutes les entrées Redo en relation avec ses blocs soient écrites sur les Log Files. Cette situation ne peut se produire que si les modifications ne sont pas confirmées par un COMMIT (car sinon le **LGWR** aurait été déclenché et les entrées Redo en question auraient été déjà écrites). Si le **DBWn** sait qu'il va écrire des **blocs dirty** non confirmés (dont les entrées Redo ne sont pas écrites sur disque), il le signale au **LGWR** et attend un message de ce dernier lui signalant la fin de sa tâche (écriture sur les Log Files des entrées redo en relation avec les blocs **dirty**) ; c'est à ce moment là que le **DBWn** aura le feu vert pour écrire les blocs en question.

### Processus LGWR (Log Writer)



LGWR écrit dans les cas suivants :

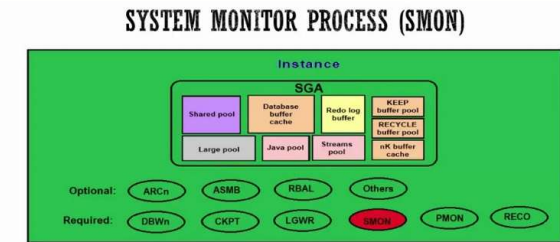
- validation
- un tiers du cache est occupé
- la journalisation atteint 1 Mo
- toutes les trois secondes
- avant que le processus DBWn ne procède à une opération d'écriture

- Lorsque le **LGWR** ne peut pas écrire sur un fichier du groupe courant (endommagement ou indisponibilité du fichier), il le fait sur les autres fichiers du même groupe et signale une erreur sur les **fichiers de trace** du système. Mais si tous les fichiers du groupe sont inaccessibles, **LGWR** n'écrit nulle part et le signale aussi au système.
- Enfin, il est à noter qu'Oracle affecte un code appelé **SCN (System Change Number)** à toute **transaction validée** (COMMIT). Ce code est inscrit par le **LGWR** sur les entrées Redo dans les **fichiers de journalisation**. Ce code est très important dans le mécanisme de récupération des données en cas de crash du système.

# Instance Oracle : Processus → Processus SMON (System Monitor)

## Processus SMON (System Monitor) :

- En cas d'échec de l'instance Oracle, les informations présentes dans la mémoire SGA qui n'ont pas été écrites sur disque sont perdues. La défaillance du système d'exploitation, par exemple, provoque l'échec de l'instance. Lorsqu'une instance a échoué, le processus d'arrière-plan **SMON** la **recupère automatiquement** lors de la **réouverture** de la base de données.
- Au cours d'une récupération, **SMON** effectue un **roll forward** qui consiste à enregistrer sur les fichiers de données les **modifications confirmées** qui n'ont pas été écrites par le **DBWn**, et un **roll back** pour annuler les **modifications non confirmées** écrites par le **DBWn** sur les fichiers de données.
- La récupération d'une instance implique les étapes suivantes :
  - Réimplémenter les modifications** pour récupérer les données non enregistrées dans les fichiers de données, mais enregistrées dans le **fichier de journalisation en ligne (online)**. Ces données n'ont pas été enregistrées sur disque du fait de l'effacement de la mémoire SGA suite à l'échec de l'instance. Lors de cette opération de réimplémentation, le processus **SMON** lit les fichiers de journalisation et applique aux blocs de données les modifications qui y sont enregistrées. Etant donné que toutes les **transactions validées** ont été enregistrées dans les fichiers de journalisation, ce processus permet de récupérer **ces transactions** dans leur **intégralité**.



- Performs recovery at instance startup
- Cleans up unused temporary segments

# Instance Oracle : Processus → Processus SMON (System Monitor)

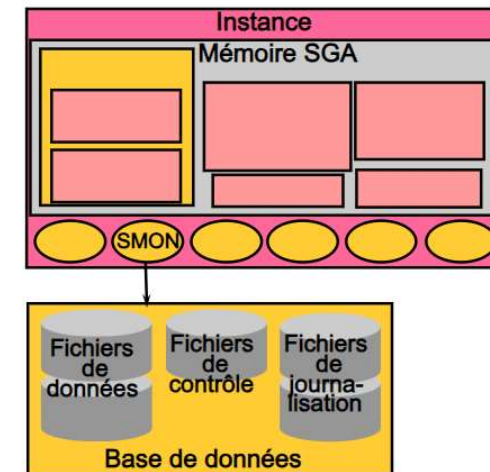
## Processus SMON (System Monitor) :

- 2) **Ouvrir la base de données** pour permettre aux utilisateurs de s'y connecter. Les données qui ne sont pas **verrouillées** par des **transactions non récupérées** sont immédiatement disponibles.
- 3) **Annuler les transactions non validées**. Ces transactions sont annulées (*rollback*) par le processus **SMON** ou par les **processus serveur** lorsqu'ils accèdent aux données verrouillées.

Le processus **SMON** exécute également des fonctions de gestion de l'espace :

- 4) Il combine ou **fusionne les espaces libres** adjacents dans les fichiers de données.
- 5) Il **libère les segments temporaires** et augmente ainsi **l'espace disponible** dans les fichiers de données.

## Processus SMON (System Monitor)



### Responsabilités :

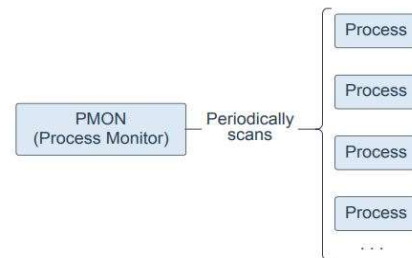
- Récupération de l'instance :
  - réimplémente des modifications dans les fichiers de journalisation,
  - ouvre la base de données pour permettre l'accès aux utilisateurs,
  - annule les transactions non validées.
- Fusion de l'espace libre
- Libération des segments temporaires segments

Si la récupération de certaines transactions échoue à cause de l'indisponibilité d'un tablespace ou d'un fichier, SMON attend qu'ils soient rétablis et récupèrent les transactions en question.

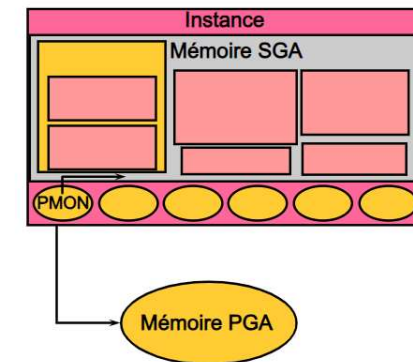
# Instance Oracle : Processus → Processus PMON (Process Monitor)

## Processus PMON (Process Monitor) :

- **PMON** (*Process Monitor*) est le processus responsable de la **récupération** lors de l'échec d'un **processus utilisateur**. Lorsqu'un processus utilisateur **plante**, **PMON annule** la transaction de ce dernier et **libère** les verrous posés sur les données modifiées à cause de la transaction en cours.
- **PMON** libère des verrous détenus par des processus en **échec** ou **terminés**.
- **PMON** assure **redémarrage** des processus de **répartiteur** et de **serveur partagé** ayant **échoué**.
- Le processus **PMON** se réveille **régulièrement** pour vérifier si l'une de ces opérations est requise. De plus, d'autres processus en arrière-plan peuvent réveiller **PMON** s'ils nécessitent l'un de ces services.



## Processus PMON (Process Monitor)

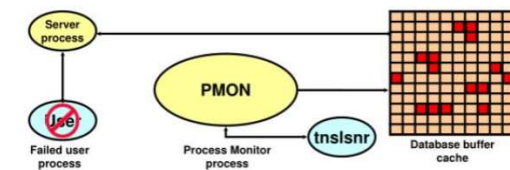


Suite à l'échec de processus, PMON exécute des opérations de nettoyage :

- annule la transaction
- libère des verrous
- libère d'autres ressources
- redémarre les répartiteurs interrompus

## Process Monitor Process (PMON)

- Performs process recovery when a user process fails
  - Cleans up the database buffer cache
  - Frees resources that are used by the user process
- Monitors sessions for idle session timeout
- Dynamically registers database services with listeners

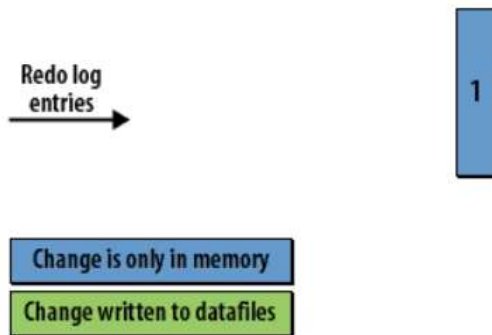




# Instance Oracle : Processus → Processus CKPT (Checkpoint)

## Processus CKPT (Checkpoint) :

- Le **processus de point de reprise CKPT** enregistre des données dans les **fichiers de contrôle/données** toutes les **trois secondes** pour identifier l'emplacement de **départ de la récupération** dans le **fichier de journalisation**. Cette opération est appelée **point de reprise**.
- Elle vise à garantir que toutes les mémoires tampon du cache de tampons de la base de données qui ont été **modifiées avant un point** dans le temps ont été écrites dans les fichiers de données. Ce point dans le temps (**position du point de reprise**) détermine le **début de la récupération de la base de données** en cas **d'échec d'une instance**. Dans ce cas, le processus **DBWn** a déjà écrit tout le contenu des **mémoires tampon du cache** de la base modifiées **avant ce point** dans le temps.
- Toutes les **3 secondes**, le processus **Checkpoint** détermine la **première entrée de journalisation** pour laquelle les modifications n'ont pas été **écrites dans la base de données**. Cela devient le **point de contrôle** et il est enregistré dans le **fichier de contrôle** et dans tous les **fichiers de données**. La série d'images suivante illustre ce processus :



- 1) Au fur et à mesure que des modifications sont apportées à la base de données, elles sont rapidement enregistrées dans le **journal redo**, mais ne sont pas **immédiatement** écrites dans les **fichiers de données**.

# Instance Oracle : Processus → Processus CKPT (Checkpoint)

## Processus CKPT (Checkpoint) :



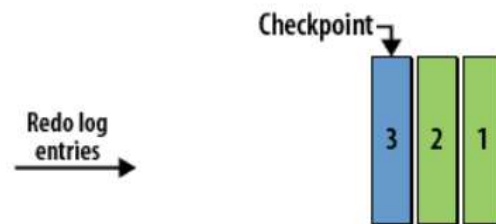
Change is only in memory  
Change written to datafiles

- 2) Nous avons **trois entrées** de journalisation. Ils sont tous affichés en **bleu**, car **DBWR** n'a encore écrit aucune des modifications apportées aux fichiers de données.



Change is only in memory  
Change written to datafiles

- 3) Le processus **DBWR** écrira certaines modifications. Ici, les modifications pour les **entrées 1 et 2** ont été écrites dans les fichiers de données.

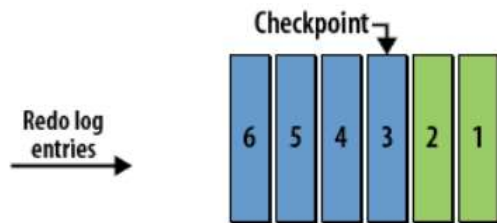


Change is only in memory  
Change written to datafiles

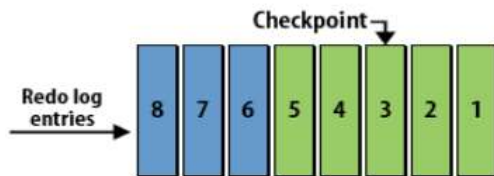
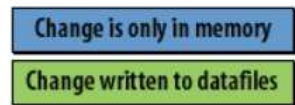
- 4) Un **point de contrôle** est enregistré toutes les **trois secondes**. Ici, le **point de contrôle** est l'entrée de **journalisation 3**, car toutes les **modifications** antérieures **ont été écrites**.

# Instance Oracle : Processus → Processus CKPT (Checkpoint)

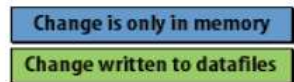
## Processus CKPT (Checkpoint) :



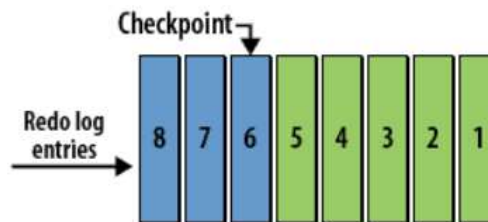
5) Ce processus se poursuit. D'autres enregistrements de rétablissement sont écrits.



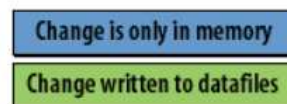
6) Plus de modifications sont écrites dans les fichiers de données.



...more changes are written to the datafiles...



7) Le point de contrôle est avancé.





# Instance Oracle : Processus → Processus CKPT (Checkpoint)

## Processus CKPT (Checkpoint) :

- ❖ Lorsqu'un événement *checkpoint* (*point de vérification*) se produit, tous les blocs *dirty* doivent être écrits sur disque ; c'est toujours le **DBWn** qui s'en charge à la suite d'un message envoyé par **CKPT** au **DBWn**.
- ❖ Après écriture des **blocs modifiés**, le **CKPT** écrit le **SCN** de la **dernière transaction confirmée** sur les entêtes (*headers*) des **fichiers de données** ainsi que dans le **fichier de contrôle** ; ces deux derniers sont dits synchronisés. En cas de crash du système, le **point de reprise** est défini par ce **SCN** et Oracle récupère à partir des fichiers de journalisation toutes les transactions qui ont été confirmées après le **SCN** inscrit sur le fichier de contrôle/entêtes de fichiers de données.

Le *checkpoint* se produit dans les cas suivants :

- ❖ La *synchronisation/ checkpoint* se produit lorsque le **LGWR** bascule d'un fichier journal à un autre. Le **LGWR** ne se permet pas de basculer de fichier journal, et donc d'écraser des **entrées Redo**, sans que le **DBW** n'écrit les blocs *dirty* du DB buffer cache, le cas échéant, le **LGWR** pourrait écraser des entrées Redo relatives à des blocs modifiés non encore écrits sur disque.
- ❖ La *synchronisation/ checkpoint* se produit aussi lors de la **fermeture** de la base de données ou lors de la **mise hors ligne** ou en **READ ONLY** d'un **tablespace**.
- ❖ Le *checkpoint* peut être forcé à travers la requête : **ALTER SYSTEM CHECKPOINT**

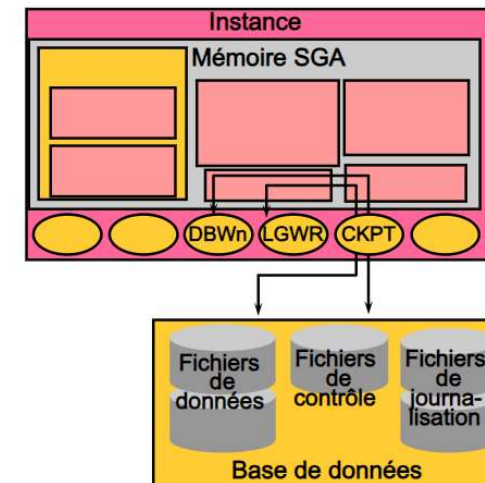
# Instance Oracle : Processus → Processus CKPT (Checkpoint)

## Processus CKPT (Checkpoint) :

❖ Les **points de reprise** sont mis en oeuvre pour les raisons suivantes :

- Pour garantir que les **blocs de données** modifiés qui se trouvent en mémoire sont **régulièrement écrits** sur disque afin d'éviter une perte des données en cas de panne du système ou de la base de données.
- Pour **réduire le temps de récupération** d'une instance. Seules les entrées de journalisation postérieures au dernier point de reprise doivent être traitées pour que la récupération ait lieu.
- Pour garantir que toutes les **données validées** ont été écrites dans les fichiers de données lors de **l'arrêt**.

## Processus CKPT (Checkpoint)

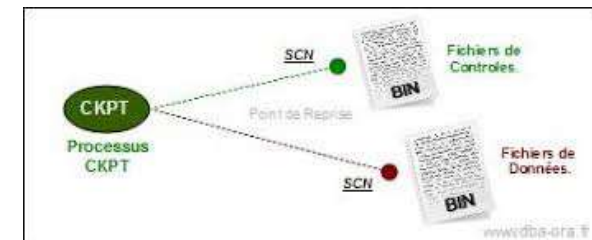


Ce processus est chargé :

- de signaler DBWn aux points de reprise,
- de mettre à jour les en-têtes de fichiers de données avec les informations sur le point de reprise,
- de mettre à jour les fichiers de contrôle avec les informations sur le point de reprise.

❖ Le processus **CKPT** écrit les informations de point de reprise suivantes : la **position du point de reprise**, le numéro **SCN** (System Change Number), **l'emplacement de départ de la récupération** dans le fichier de journalisation, des informations sur les fichiers de journalisation, etc.

**Remarque** : Le processus CKPT n'écrit pas de blocs de données sur le disque ou de blocs de journalisation dans les fichiers de journalisation en ligne.





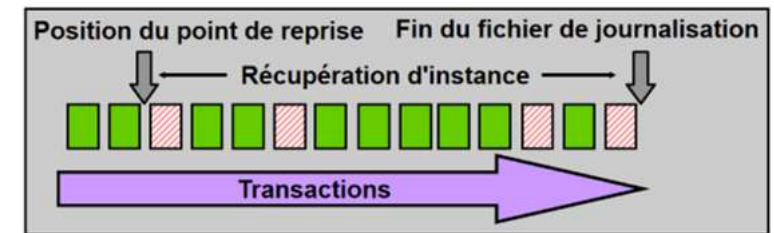
# Régler la récupération d'instance

## Régler la récupération d'instance

- Les informations relatives aux transactions sont toujours enregistrées dans les **groupes de fichiers de journalisation** avant que l'instance ne renvoie **commit complète** pour une **transaction**.
- Les informations des **groupes de fichiers de journalisation** garantissent que la **transaction** pourra être récupérée en **cas d'échec**. Ces mêmes informations de transaction doivent également être écrites dans le **fichier de données**.
- L'écriture dans le fichier de données a généralement lieu un peu après l'enregistrement des informations dans les groupes de fichiers de journalisation, car le **processus d'écriture** dans les fichiers de données est beaucoup plus lent que les écritures de journalisation (les **écritures aléatoires** dans les fichiers de données sont **plus lentes** que les **écritures en série** dans les fichiers de journalisation).

## Régler la récupération d'instance

- Au cours de la récupération d'instance, les transactions entre la position du point de reprise et la fin du fichier de journalisation doivent être appliquées aux fichiers de données.
- Réglez la récupération d'instance en contrôlant la différence entre la position du point de reprise et la fin du fichier de journalisation.



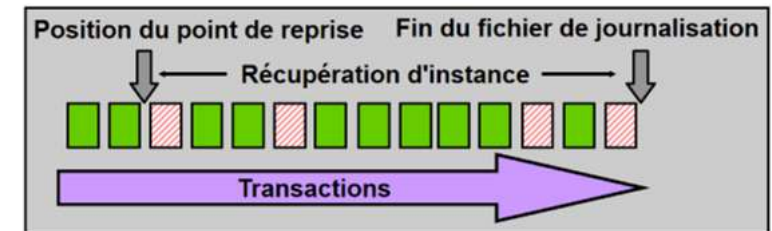
# Régler la récupération d'instance

## Régler la récupération d'instance

- Pour effectuer le suivi des informations déjà écrites dans les fichiers de données, la base de données utilise des **points de reprise (checkpoints)**. Un **point de reprise garantit** qu'au moment où il se produit, toutes les données jusqu'à un certain numéro SCN sont enregistrées dans le fichier de données. Les transactions qui ont lieu après la position du point de reprise peuvent ou non avoir déjà été écrites dans le fichier de données approprié. Dans le schéma de la diapositive ci-contre, les blocs hachurés n'ont pas encore été écrits sur le disque.
- Le **temps nécessaire à la récupération** de l'instance correspond au temps nécessaire pour rétablir les fichiers de données de leur état au moment du **dernier point de reprise** jusqu'à l'état correspondant au dernier numéro SCN enregistré dans le **fichier de contrôle**. L'administrateur contrôle cet instant en définissant une durée **MTTR** cible (en **secondes**) ainsi que la taille des groupes de fichiers de journalisation.
- La **distance** entre la position du **point de reprise** et la fin du groupe de fichiers de journalisation ne peut jamais être **supérieure à 90 %** du groupe de fichiers de journalisation le plus petit.

## Régler la récupération d'instance

- Au cours de la récupération d'instance, les transactions entre la position du point de reprise et la fin du fichier de journalisation doivent être appliquées aux fichiers de données.
- Réglez la récupération d'instance en contrôlant la différence entre la position du point de reprise et la fin du fichier de journalisation.



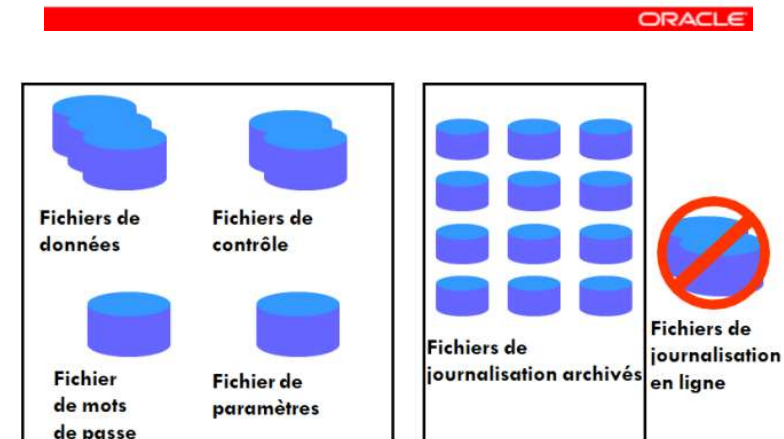
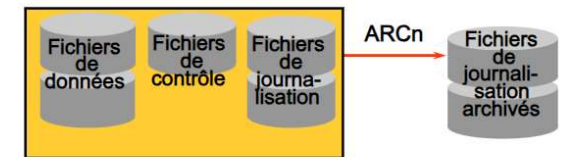
# Instance Oracle : Processus → Processus ARCn (Processus d'archivage)

## Processus ARCn (Processus d'archivage) :

- Tous les autres processus d'arrière-plan sont **facultatifs**, selon la configuration de la base de données. Toutefois, l'un d'entre eux, **ARCn**, joue un rôle essentiel dans la récupération d'une base suite à une **défaillance du disque**.
- Le serveur Oracle passe d'un fichier de journalisation en ligne au suivant à mesure que ceux-ci se remplissent. Le passage d'un fichier de journalisation à un autre est un changement de fichier de journalisation.
- Le processus **ARCn** lance la sauvegarde ou l'archivage du groupe de fichiers de journalisation remplis à chaque changement de fichier de journalisation. Il **archive automatiquement** le fichier de journalisation en ligne pour qu'il puisse être réutilisé.
- Ainsi, toutes les **modifications** apportées à la base sont **conservées**. Ceci permet au DBA de récupérer la base de données jusqu'au point de panne, même si un disque est endommagé.

## Processus ARCn (processus d'archivage)

- Processus d'arrière-plan facultatif
- En mode **ARCHIVELOG**, il archive automatiquement les fichiers de journalisation en ligne
- Il enregistre toutes les modifications apportées à la base de données



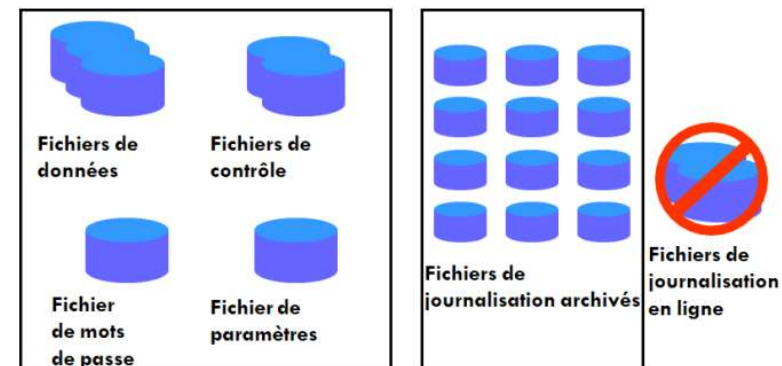


# Instance Oracle : Processus → Processus ARCn (Processus d'archivage)

## Processus ARCn (Processus d'archivage) :

### Archiver les fichiers de journalisation :

- L'une des principales décisions de l'administrateur consiste à déterminer s'il doit configurer la base de données pour un fonctionnement en mode **ARCHIVELOG** ou en mode **NOARCHIVELOG**.
  - **Mode NOARCHIVELOG** : Dans ce mode, les fichiers de journalisation en ligne sont écrasés à chaque changement de fichier de journalisation. Le processus **LGWR** n'écrase pas un groupe de fichiers de journalisation tant que le point de reprise du groupe n'est pas terminé. Ceci garantit la récupération des données validées en cas d'échec d'une instance. Dans ce cas, seules les données de la mémoire SGA sont perdues (aucune donnée des disques n'est perdue). Une défaillance du système d'exploitation, par exemple, provoque la défaillance de l'instance.
  - **Mode ARCHIVELOG** : Si la base de données fonctionne en mode **ARCHIVELOG**, les groupes **inactifs** de **fichiers de journalisation en ligne remplis** doivent être **archivés** pour pouvoir être **réutilisés**. Etant donné que les modifications de la base sont enregistrées dans les fichiers de journalisation en ligne, l'administrateur peut utiliser la **sauvegarde physique** des **fichiers de données** et les **fichiers de journalisation en ligne archivés** pour récupérer la base sans perdre les données validées en cas de panne en un point unique, notamment de **perte d'un disque**. En règle générale, une **base de données de production est configurée pour fonctionner en mode ARCHIVELOG**.





## Instance Oracle : Processus → Autres processus Oracle

- ❑ **CJQn** est le processus coordinateur qui exécute les **tâches planifiées** par les utilisateurs. Ces tâches sont principalement des programmes **PL/SQL planifiés** entre autres grâce au package **DBMS\_SCHEDULER**. Les informations concernant les tâches (jobs) planifiées d'une base de données sont stockés dans la table **DBA\_SCHEDULER\_JOBS** du **dictionnaire de données**. Le processus **CJQn** peut exécuter un grand nombre de tâches grâce à ses processus esclaves (J000 à J999).
- ❑ **MMAN** (*Memory Manager*) est un processus introduit par *Oracle 10g* qui gère les tailles des composantes de la SGA lorsque l'ASMM est activée.
- ❑ **MMON** (*Memory Monitor*) est un processus introduit par *Oracle 10g*. Son objectif est de collecter **des statistiques** sur la **SGA**. MMON est déclenché toutes les **60 minutes** pour collecter des statistiques à partir des **vues dynamiques** et **statiques** du **dictionnaire de données** et les stocker dans **des tables** de la base de données appelées **Automatic Workload Repository (AWR)**. Le propriétaire de ces tables est **SYSMAN** et sont stockées dans le **tablespace SYSAUX**. Pour activer l'AWR, il faut instancier le paramètre **STATISTICS\_LEVEL**. La valeur assignée à ce paramètre détermine le **niveau de statistiques** que le **MMON** doit collecter ; **BASIC** veut dire que l'AWR est désactivé (ce qui désactive plusieurs options de monitoring automatique) et que peu de statistiques sont collectées. **TYPICAL** est le niveau standard, il s'agit de la valeur par défaut du paramètre. **ALL** permet la collecte des statistiques du niveau **TYPICAL** avec en plus les plans d'exécution ainsi que d'autres informations reliées au système d'exploitation.





## Instance Oracle : Processus → Autres processus Oracle

- ❑ **RECO** (*Recoverer*), il s'agit d'un processus optionnel permettant de résoudre les **transactions interrompues** brutalement dans un système de bases de données.
- ❑ **Dnnnn** (*nnnn* représente une suite de nombre entiers) : Ce processus permet de router les requêtes distants vers les autres serveurs dans l'architecture BD distribuée.
- ❑ **Snnnn** : Ce processus permet de recevoir les demandes de connexions distantes envoyées par le processus **Dnnnn** d'un serveur distant.
- ❑ **LCKn** (*Lock*) est un processus de verrouillage utilisé lorsque Oracle Parallel Server est installé.
- ❑ **Listener** : Pour se connecter à distance à une base de données, il faut établir une connexion avec un process qui gère la relation entre votre client et l'instance : dans le cas le plus général, cette connexion est établie avec un **process serveur dédié**. Ce process est un processus sous Unix et un thread sous Windows. Il est possible d'utiliser un process partagé pour gérer le dialogue avec l'instance, mais cela sort du scope de cette explication. Pour arriver à établir la connexion, il faut le demander à quelqu'un. Le **listener** peut être ce vecteur: il sert **d'agent de connexion**. Tout serait trop simple, s'il n'existait pas d'autres agents de connection comme **Oracle Connection Manager (CMAN)** dans certaines configurations plus évoluées.



## Instance Oracle : Processus → Autres processus Oracle

❑ **Listener (suite)** : L'un des rôles du listener est donc de créer \*\*sous Unix et Linux, on dit spawner\*\*, le process serveur qui va dialoguer avec votre client et de vous mettre en relation avec ce process. Pour cela, il se met à l'écoute sur une **adresse réseau** sur un **port particulier** et répond à toutes les demandes de connexions. Le nom du **processus listener** est **tnslsnr** ou **tnslsnr.exe**. Pour configurer le listener, il faut créer un fichier **listener.ora** dans le répertoire pointé par les variables **ORACLE\_HOME/network/admin** ou **TNS\_ADMIN** si cette variable existe. Il est à noter que si ce fichier n'existe pas, vous pouvez démarrer malgré tout un **listener** en 10g. Dans ce cas, il se mettra en écoute sur le **port 1521** sur le réseau de votre machine. Pour créer un fichier listener.ora, le moyen le plus simple est d'utiliser un des outils de configuration du réseau : **network configuration assistant (netca)**, **network manager (netmgr)** ou **Enterprise Manager 10g**.

- Pour démarrer le listener, tapez : **lsnrctl start listener** où listener est le nom de votre listener.
- Pour vérifier l'état de votre listener (les adresses et les ports sur lesquelles il écoute), tapez : **lsnrctl status listener**

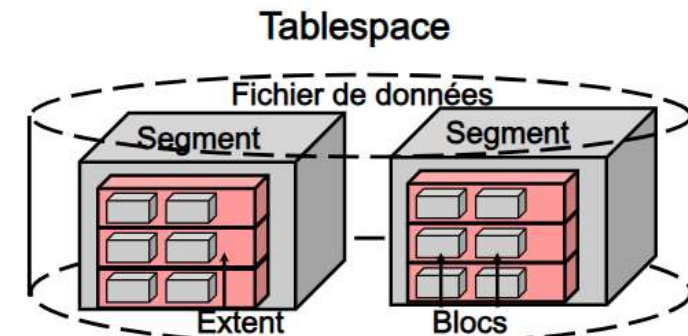
Il faut savoir que le nombre maximum de processus exécutés en même temps dans l'instance est contrôlé par le paramètre d'initialisation **PROCESSES**. La vue **V\$PROCESS** inclut les informations de tous les processus en cours d'exécution dans l'instance.

# Structure logique

- La hiérarchie de la structure logique se présente comme suit :
  - Une **base de données Oracle** contient au moins un **tablespace**.
  - Un **tablespace** comporte un ou **plusieurs segments**.
  - Un **segment** est composé d'**extents**.
  - Un **extent** est composé de **blocs Oracle contigus/adjacents**.
  - Un **bloc** représente la **plus petite unité** sur laquelle portent les opérations de lecture et d'écriture.
  
- L'architecture d'une base de données Oracle contient les structures logiques et physiques qui constituent la base de données.
  - La **structure physique** comprend les fichiers de contrôle, les fichiers de journalisation en ligne (online redo log) et les fichiers de données qui constituent la base de données.
  - La **structure logique** est composée de tablespaces, de segments, d'extents (ensemble de blocs contigus) et de blocs de données.
  
- Le **serveur Oracle** permet un contrôle précis de l'utilisation de l'espace disque à l'aide de tablespaces et de structures de stockage logiques constituées de segments, d'extents et de blocs de données.

## Structure logique

- La structure logique définit le mode d'utilisation de l'espace physique d'une base de données.
- Cette structure possède une hiérarchie composée de tablespaces, de segments, d'extents et de blocs.



# Structure logique

## Tablespaces (structures logiques) :

Les **données d'une base de données Oracle** sont stockées dans des **tablespaces**.

- Une **base de données Oracle** peut être divisée en plus petites zones logiques d'espace appelées **tablespaces**.
- Un **tablespace** ne peut appartenir qu'à **une seule base de données** à la fois.
- Chaque **tablespace** est constitué **d'un ou de plusieurs fichiers du système d'exploitation**, appelés **fichiers de données**.
- Un **tablespace** peut contenir **un ou plusieurs segments**.
- Les **tablespaces** peuvent être mis **en ligne** lorsque la **base de données** est **active**.
- A l'exception du tablespace **SYSTEM** ou des tablespaces contenant un **segment d'annulation actif**, les tablespaces peuvent être mis **hors ligne** sans interruption de la base de données.
- Les tablespaces peuvent être accessibles en **lecture et écriture** ou en **lecture seule**.

## Fichiers de données (structure non logique > structures physiques) :

- **Chaque tablespace** d'une base de données Oracle contient un ou plusieurs fichiers appelés **fichiers de données**. Ces fichiers sont des structures physiques conformes au système d'exploitation sur lequel s'exécute le serveur Oracle.
- Un **fichier de données** ne peut appartenir qu'à un **seul tablespace**.
- Un serveur Oracle crée un fichier de données dans un tablespace en allouant l'espace disque défini et un petit espace supplémentaire.
- L'administrateur de base de données peut **modifier la taille** des **fichiers de données** après leur création ou indiquer que leur taille peut augmenter de **façon dynamique** à mesure que celle des **objets du tablespace s'accroît**.

# Structure logique

## Segments :

- Un **segment** est un espace alloué à une structure de stockage logique spécifique **d'un tablespace**.
- Un **tablespace** peut contenir **un ou plusieurs segments**.
- Un segment ne peut pas être réparti sur plusieurs tablespaces, mais peut s'étendre à plusieurs fichiers de données d'un même tablespace.
- Chaque **segment** est constitué **d'un** ou de **plusieurs extents**.

## Extents :

Les extents permettent d'allouer de l'espace à un segment.

- Un **segment** peut être constitué **d'un ou de plusieurs extents**.
  - Lorsque vous créez **un segment**, celui-ci contient au **moins un extent**.
  - A mesure que la **taille du segment augmente**, des **extents** lui sont **ajoutés**.
  - L'administrateur de base de données peut **ajouter manuellement** des **extents** à un segment.
- Un **extent** est constitué de **blocs Oracle contigus**.
- Un **extent** ne peut pas s'étendre sur plusieurs fichiers de données. Il doit donc appartenir à un **seul fichier de ce type**.

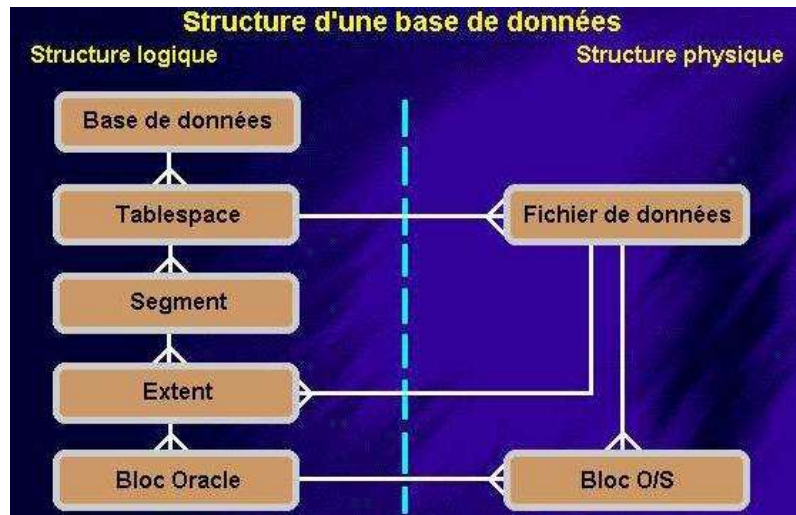


# Structure logique

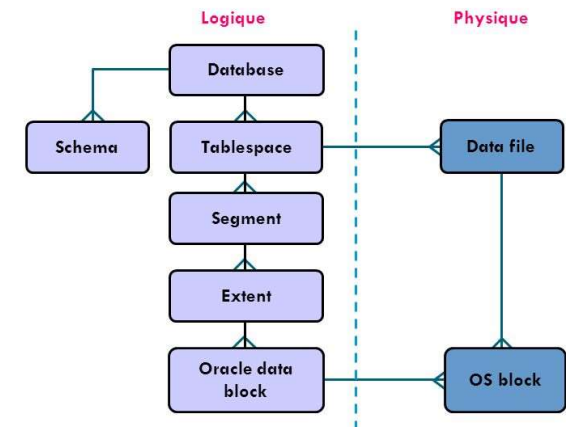
## Blocs de données :

Le serveur Oracle gère l'espace de stockage des fichiers de données à l'aide d'unités appelées **blocs de données ou blocs Oracle**.

- Lorsque le degré de détail maximum est atteint, les données d'une base Oracle sont stockées dans des blocs de données.
- Les **blocs de données Oracle** représentent la **plus petite unité de stockage** que le serveur Oracle peut allouer, écrire ou lire.
- Un **bloc de données** correspond à **un ou plusieurs blocs du système d'exploitation** alloués à partir d'un fichier de données existant.
- Le paramètre d'initialisation **DB\_BLOCK\_SIZE** permet de définir la taille de bloc standard lors de la création d'une base de données Oracle.
- La **taille des blocs de données** doit correspondre à **un multiple de la taille des blocs** du système d'exploitation afin d'éviter les opérations d'entrée/sortie inutiles.
- La taille maximale d'un bloc de données dépend du système d'exploitation utilisé.



## Structure logique



# Traiter les instructions SQL

## Pour exécuter une requête SELECT

- Nous supposons que l'**instance** a déjà **démarré** et que la connexion entre un **processus utilisateur** et un **processus serveur** est **établie**. Le processus utilisateur envoie une requête **SELECT**, voici en gros les étapes d'exécution :

**1) Phase de parse** : Le **processus serveur** vérifie si la requête existe dans le **Library Cache** du **Shared Pool**. Si elle n'y existe pas, une **analyse syntaxique** est effectuée (vérification de l'exactitude des mots réservés, de leur ordre etc.) suivie par une **analyse sémantique** qui consiste à (1) vérifier l'existence des tables et des colonnes consultées dans la requête via le dictionnaire de données et (2) à vérifier les privilèges de l'utilisateur en cours. L'analyse sémantique est effectuée à partir du **Dictionary Cache** si les informations en question y existent, sinon Oracle les y charge à partir du dictionnaire de données. Par la suite, un **plan d'exécution** de la requête est calculé. Ce *parse* est appelé « *hard parse* » puisqu'il consomme beaucoup de ressources comparé à un « *soft parse* » qui se produit dans le cas où la requête est trouvée dans le **Library Cache**. Dans ce dernier cas, seule la vérification des privilèges de l'utilisateur est effectuée et toutes les autres opérations (vérification syntaxique et calcul du plan d'exécution) sont épargnées.

## Traiter les instructions SQL

- Connexion à une instance via :
  - le processus utilisateur,
  - le processus serveur.
- Les composants du serveur Oracle utilisés dépendent du type d'instruction SQL :
  - Les interrogations renvoient des lignes.
  - Les instructions LMD consignent les modifications.
  - La validation garantit la récupération de la transaction.
- Certains composants du serveur Oracle n'interviennent pas dans le traitement des instructions SQL.

# Structure logique Traiter les instructions SQL

## Pour exécuter une requête SELECT (suite) :

**2. Phase d'exécution** : Une fois le **plan d'exécution** calculé, le processus serveur **exécute** et vérifie si les blocs de données cibles existent dans le **DB Buffer Cache**. Si ce n'est pas le cas, il les **charge** à partir des **fichiers de données**.

**3. Phase de fetch** : Une fois le processus serveur **récupère** les données, il les trie si la requête inclut la clause **ORDER BY** puis il les renvoie au processus utilisateur.

## Pour exécuter une requête UPDATE

**1. Phase de parse** : La même que pour une requête SELECT.

**2. Phase d'exécution** : Le processus serveur charge dans le **DB Buffer Cache** les blocs de **données à modifier** si elles n'y sont pas déjà et **verrouille** les **lignes à modifier**. Par la suite, le **Redo Entry** correspondant à la requête (incluant entre autres l'image avant et après des données) est stocké dans le **Redo Log Buffer**. On stocke aussi l'image de données avant modification dans un segment UNDO (annulation) situé dans le tablespace **UNDO**. Finalement les blocs de données sont modifiés dans le **DB Buffer Cache**, ils seront écrits de manière différée sur le disque par le **DBWn**.

**3. Phase de fetch** : il n'y a pas d'opération **fetch** dans le cas d'une requête DML, l'utilisateur reçoit juste un message concernant le déroulement de l'opération.



# Structure logique Traiter les instructions SQL

## Lors d'un COMMIT

Le **LGWR** assigne un **SCN (System Change Number)** à la transaction validée et écrit par la suite toutes les entrées Redo de la transaction confirmée dans les **fichiers de journalisation** à partir du **Redo Log Buffer**. Sur disque et plus précisément sur les **segments d'annulation**, les informations en relation avec la **transaction validée** sont **libérées**.

## Lors d'un ROLLBACK

Lorsque l'utilisateur effectue un **ROLLBACK**, il existe deux scénarii possibles :

- 1.** Les **blocs modifiés** sont déjà **inscrits** dans les **fichiers de données** ; dans ce cas, la récupération de l'ancienne image se fait à partir du **segment d'annulation**.
- 2.** La **modification** est **encore en mémoire (DB Buffer Cache)** et n'a pas été écrite sur disque. **L'ancienne image** des données existe donc dans les fichiers de données, et **l'annulation** est beaucoup **plus rapide** que dans le premier cas.